

Summer Training Program WEEK 2 - DAY 8

Object oriented programming structure

It allows us to make customized data structures. It is a very important concept for actual industrial backend development.

Principle of oops:

1. Think of an object
2. Draw the object
3. From the drawing, create the object

What are objects and classes?

In real life,

1. Object: anything in existence
2. Class: textual or visual(architecture/blueprint) representation of an object

In context to computer science,\

1. Object: it is a storage container (homogeneous or heterogeneous)
2. Class: it is the textual representation of the object, i.e. how the object is represented in the memory(the container).

How to actually make objects:

>>A real industrial scenario is how the client gives a verbal/textual/visual representation of what kind of a solution he wants, so we take an example,

Eg: the client wants us to build a food delivery app

Step1: identify the objects

1. Restaurant:
Attributes:
name, phone, email, address, operating_hours, rating, price_per_person, menu
2. Menu
name, type, dishes[]
3. Dish
name, price, category, rating

4. User
name, phone, email, address[] , gender, profile_image
5. Address
address_line, city, state, pincode, tag
6. DeliveryAgent
name, phone, email, driving_licence, vehicle, work_hours
7. Vehicle
registration_number, brand, model, year_of_registration, color
8. Order
id, amount, dishes, ?, ? date, time

Relationship mapping of objects:

Objects may be related to each other, this is called mapping the relationships. Relationships may be of 3 types

1. One to many
2. Many to one
3. Many to many

Eg: one menu has many dishes (1 to *)
One restaurant has one menu (1 to 1)
many restaurant can have many menus [in case of programming a SaaS type solution]

Step2: represent the object

```
class object_name:  
    pass
```

This is how we define an object, class is nothing but the way of definition of an object or the representation of an object. We code the class to represent the object.

We will represent 3 of the classes first so,

Restaurant

Menu

Dish

```

5  ✓ class Restaurant:
6      |     pass
7
8  ✓ class Menu:
9      |     pass
10
11 ✓ class Dish:
12     |     pass

```

Now we will start to code class dish first, this is because a restaurant has a menu (relationship) and then a menu has many dishes (relationship) but the dishes are not further dependent on anything.

First of all we code a constructor:

```

12  ✓ def __init__(self):
13      |     pass

```

‘__init__’: its the constructor and also a special variable called dunder. Every class has a constructor which has this same name.

Create objects:

We can make objects as such:

```

16  restaurant = Restaurant()
17
18  menu = Menu()
19
20  dish1 = Dish()
21  dish2 = Dish()
22  dish3 = Dish()

```

If we print these objects:

```

24  print('restaurant:', restaurant)
25  print('menu:', menu)
26
27  print('dish1:', dish1)
28  print('dish2:', dish2)
29  print('dish3:', dish3)

```

OUTPUT:

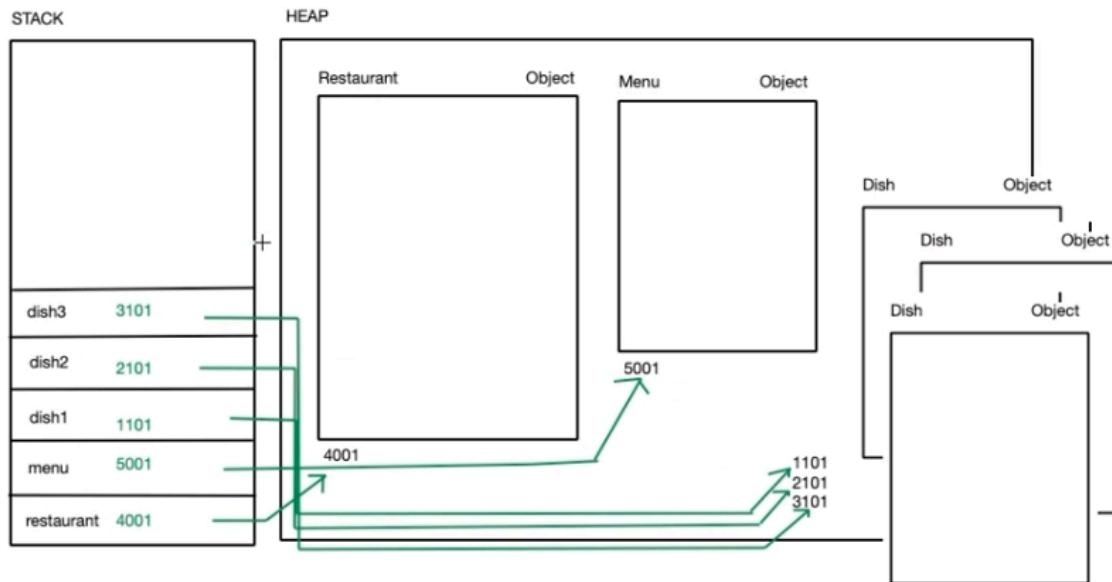
```

restaurant: <__main__.Restaurant object at 0x00000227F5B66A50>
menu: <__main__.Menu object at 0x00000227F5B66BA0>
dish1: <__main__.Dish object at 0x00000227F5B66CF0>
dish2: <__main__.Dish object at 0x00000227F5DF8A50>
dish3: <__main__.Dish object at 0x00000227F5DF8B90>

```

All of these objects are created in main thread by default

Memory representation:



Right now all the objects are empty

We can use the vars() function to print the data in the objects:

```
31 print('data in restaurant:', vars(restaurant))
32 print('data in menu:', vars(menu))
33 print('data in dish1:', vars(dish1))
34 print('data in dish2:', vars(dish2))
35 print('data in dish3:', vars(dish3))
```

OUTPUT:

```
data in restaurant: {}
data in menu: {}
data in dish1: {}
data in dish2: {}
data in dish3: {}
```

Adding variables to the object: (assigning values to attributes)

We use the dot (membership) operator for this purpose.

```
11 dish1 = Dish()
12 dish1.name = 'Noodles'
13 dish1.price = 300
14 dish1.category = 'Chinese Veg'
15 dish1.rating = 4.5
16
17 dish2 = Dish()
18 dish2.name = 'Burger'
19 dish2.price = 100
20 dish2.category = 'Veg'
21 dish2.rating = 3.5
22
23 dish3 = Dish()
24 dish3.name = 'Pizza'
25 dish3.price = 400
26 dish3.category = 'Italian Veg'
27 dish3.rating = 4.5
28
29 dishes = [dish1, dish2, dish3]
30
31 menu = Menu()
32 menu.name = 'Johns Special'
33 menu.type = 'vegetarian'
34 menu.dishes = dishes # 1 to many mapping
35
36 restaurant = Restaurant()
37 restaurant.name = 'Johns Cafe'
38 restaurant.phone = '+91 99999 11111'
39 restaurant.email = 'info@johnscafe.com'
40 restaurant.address = 'redwood Shores'
41 restaurant.operating_hours = '10:00 to 23:00'
42 restaurant.rating = 4.5
43 restaurant.price_per_person = 500
44 restaurant.menu = menu # 1 to 1 mapping
```

Here the type of 'menu' is 'Menu' and the type of 'dish1-3' is 'Dish' and the type of 'restaurant' is 'Restaurant'

These are all custom objects.

All these objects are now contained with data, they are not empty, now if we use the vars() function, we won't get null output but the list of data we defined.

We can update the values of any of the attributes
We can even delete the attributes from the object.
We can also add data at any time.

```
48 restaurant.price_per_person = 400
49
50 del restaurant.price_per_person
51
52 restaurant.delivery_timings = '18:00 to 20:00'
```

This is called the **open close principle** of objects in python, we can close and open the object to edit its attributes anytime.

>>Another example: make a user

```
1 class User:
2     pass
3
4 user1 = User()
```

When we write the fourth statement it means that the

1. LHS: user1
It is the reference variable, which has the hashcode of the object
It is created in the stack
2. RHS: user()
It is the actual object/container in the heap, whose hashcode is returned to the reference variable.

NOTE: This is the basic representation in the RAM, the temporary memory, when we actually map objects to the database, it is called ORM : Object Relational Mapping, which is the process of mapping the database to the actual relational database.

We can also assign one object to another.

CODE:

```
1 class User:
2     pass
3
4 user1 = User()
5 user2 = User()
6
7 print('user1: ', user1, id(user1))
8 print('user2: ', user2, id(user2))
9 print('variables in user1: ', vars(user1))
10 print('variables in user2 : ', vars(user2))
11
12
13 user3 = user1
14 print('user3: ', user3, id(user3))
```

OUTPUT:

```
user1: <__main__.User object at 0x0000012D3F3C6A50> 1293846080080
user2: <__main__.User object at 0x0000012D3F658A50> 1293848775248
variables in user1: {}
variables in user2 : {}
user3: <__main__.User object at 0x0000012D3F3C6A50> 1293846080080
```

Constructor:

Now if we want to add data to the object, we have to do it manually:

```
18 user1.name = 'John'
19 user2.full_name = 'Fionna Flynn'
20 user3.age = 20
21 user2.age = 25
22 user3.email = 'john@example.com'
```

It is inefficient to write data like this (manually, there may be spelling mistakes and stuff)
So we should standardize the process of writing data eg: fix the number of attributes, here the constructor is used.

Defining again with constructor:

- constructor function in user:

```
1 class User:
2     def __init__(self):
3         print('self:' self, type(self))
```

- Here the literal value of self can be anything
- Now, what is self? To understand this, we have used statement 3, which is the log statement:

```
self: <__main__.User object at 0x000001E7148E6A50> <class '__main__.User'>
self: <__main__.User object at 0x000001E714B78A50> <class '__main__.User'>
```

- This 'self' log statement gets printed twice, even though we have written it only once, this is because we have also made 2 objects in the same code for class User().
- NOTE:user3 is not an object, it is just a reference variable. [Editing user1 will also affect user3]

Whenever the RHS statement of line 5 is executed: the `__init__` is executed and user1 (reference variable containing hashcode) is copied to self

And when line 6 is executed, `__init__` is executed, the next object user2 which has a different hashcode is copied to self

```
5 user1 = User()
6 user2 = User()
```

Self is a reference variable which holds the hashcode of the current object [interview question, its equivalent to this in java and this pointer in c++]

We can easily infer the number of objects by seeing how many times `__init__` executes.

Eg:

The number of users who sign in = number of times `__init__` is executed.

- But if the user signs in and then uses a separate device, then `__init__` will not be executed again (case of reference copy operation)

Using constructor for inputs:[**standardizing the program**]

We can just simply write all the attributes of the object along with self and then assign them values:

```
1  class User:
2
3  def __init__(self, kuchbhi, phone, email, age, gender, address):
4
5      self.name = kuchbhi
6      self.phone = phone
7      self.email = email
8      self.age = age
9      self.gender = gender
10     self.address = address
```

Here when we write statements 5 to 10:

1. The LHS is the attribute created in the object
2. The RHS is the value we are inputting into the object (it may or may not be the same as that of the attribute name.)

Now when we do object construction:

```
13 user1 = User(kuchbhi='John',
14              phone='+91 99990 11110',
15              email='john@example.com',
16              age=20,
17              gender='male',
18              address='redwood shores')
19
20 user2 = User(kuchbhi='Fionna',
21              phone='+91 99991 11111',
22              email='fionna@example.com',
23              age=21,
24              gender='female',
25              address='country homes')
```

We can also use the reference copy operation in the same way:

```
28 user3 = user1
```

user3 is not an object but a reference variable.

OUTPUT:

```
user1: <__main__.User object at 0x000001A10FAA6A50> {'name': 'John', 'phone': '+91 99
990 11110', 'email': 'john@example.com', 'age': 20, 'gender': 'male', 'address': 'red
wood shores'}
user2: <__main__.User object at 0x000001A10FD48A50> {'name': 'Fionna', 'phone': '+91
99991 11111', 'email': 'fionna@example.com', 'age': 21, 'gender': 'female', 'address'
: 'country homes'}
user3: <__main__.User object at 0x000001A10FAA6A50> {'name': 'John', 'phone': '+91 99
990 11110', 'email': 'john@example.com', 'age': 20, 'gender': 'male', 'address': 'red
wood shores'}
```


NOTE: There is no concept of constructor(or function) overloading in python.

If we redefine the constructor, the old definition of the constructor will be deleted and the newest constructor will be considered.

Interview questions:

- What is more important? Object or constructor
- What is self in python?
- How to overload in python?

ASSIGNMENT:

- **Code your own concept with oops like done in the session.**
[pushed to repo]

GURASSIS KAUR

g.assiskaur@gmail.com